



## UNIVERSIDAD MAYOR DE SAN SIMÓN

Facultad de Ciencias y Tecnología  
Carrera de Ingeniería de Sistemas / Informática



### PROYECTO FINAL DE PROGRAMACIÓN MÓVIL

## Caja Cordes Bolivia – Sistema de Circulares

Aplicación móvil Android nativa para la difusión de circulares e instructivos

#### **Docente:**

Dr. Américo Fiorilo Lozada Ph.D.

#### **Integrante del grupo:**

Wálter Guzmán Méndez

Fecha de presentación: 5 de junio de 2026

Cochabamba – Bolivia

# Índice

---

|                                         |    |
|-----------------------------------------|----|
| 1. Resumen Ejecutivo .....              | 3  |
| 2. Análisis del Proyecto.....           | 3  |
| 3. Diseño del Proyecto .....            | 5  |
| 4. Arquitectura del Sistema .....       | 7  |
| 5. Metodología Aplicada.....            | 9  |
| 6. Seguimiento y Desarrollo Grupal..... | 9  |
| 7. Pruebas y Resultados .....           | 11 |
| 8. Conclusiones y Recomendaciones ..... | 12 |
| 9. Bibliografía .....                   | 12 |
| 10. Anexos .....                        | 13 |

## 1. Resumen Ejecutivo

---

El presente proyecto consiste en el desarrollo de una aplicación móvil nativa para Android, denominada «Caja Cordes Bolivia – Sistema de Circulares», cuyo propósito es centralizar y automatizar la difusión de circulares e instructivos institucionales hacia todos los dispositivos de los usuarios. La aplicación distingue dos perfiles: un administrador, que es el único que inicia sesión y se encarga de publicar el contenido, y los usuarios finales, que solo instalan la aplicación y reciben las circulares de forma automática, sin necesidad de registrarse.

La solución se construyó con lenguaje Kotlin y arquitectura Model-View-ViewModel (MVVM), apoyándose en los servicios de Firebase (Cloud Firestore para los datos, Cloud Storage para los archivos, Cloud Functions y Cloud Messaging para las notificaciones push). La difusión de cada circular se realiza en tiempo real y, además, se repite de manera programada cada cuatro horas durante dos días mediante WorkManager. El administrador dispone de un módulo de reportes con estadísticas de envío y visualización, así como de funciones para previsualizar, desactivar y eliminar publicaciones. El enfoque general del trabajo fue incremental: se construyó primero una versión funcional completa y luego se incorporaron mejoras a partir de la retroalimentación de uso.

## 2. Análisis del Proyecto

---

### 2.1. Objetivos

#### **Objetivo general:**

Desarrollar una aplicación móvil nativa para Android que permita difundir circulares e instructivos de forma automática, oportuna y controlada a todos los usuarios de la institución, garantizando que el contenido sea visto durante un periodo determinado.

#### **Objetivos específicos:**

- Implementar un acceso diferenciado en el que únicamente el administrador inicie sesión, mientras que los usuarios solo instalan la aplicación y visualizan el contenido.
- Permitir al administrador cargar circulares en formato de imagen (JPG, PNG) o documento PDF.
- Mostrar automáticamente cada nueva circular en todos los dispositivos y repetir su visualización cada 4 horas durante 2 días desde la publicación.
- Proveer notificaciones push instantáneas que alerten a los usuarios cuando se publica una circular, incluso con la aplicación cerrada.

- Ofrecer al administrador un módulo de reportes con la cantidad de circulares enviadas, fechas, número de visualizaciones y estado de difusión.
- Incorporar funciones de gestión: previsualizar, desactivar/activar y eliminar publicaciones, así como permitir a los usuarios hacer zoom sobre el contenido.
- Asegurar la compatibilidad desde Android 8.0 (API 26) hasta las versiones más recientes.

## **2.2. Problema o Necesidad Identificada**

La difusión de circulares e instructivos en instituciones suele realizarse por canales dispersos (correo, mensajería, papel o avisos físicos), lo que genera problemas recurrentes: el mensaje no llega a todos, no hay certeza de que haya sido visto, no existe un registro de su alcance y la información importante se diluye entre otras comunicaciones. Esto provoca desinformación, repetición de avisos y falta de trazabilidad. La necesidad identificada es contar con un canal único, propio y controlado que asegure que cada circular llegue a todos los dispositivos, se muestre de forma destacada y repetida durante un periodo, y deje un registro medible de su difusión y visualización.

## **2.3. Contexto**

La aplicación está pensada para un entorno institucional (Caja Cordes Bolivia) en el que un área administrativa necesita comunicar disposiciones, instructivos y circulares al personal. El servicio telemático opera sobre dispositivos móviles Android conectados a internet, con un backend en la nube que centraliza la información.

### **Público objetivo:**

- Administrador: personal autorizado del área de comunicación/administración, responsable de publicar las circulares.
- Usuarios finales: empleados de la institución que reciben y consultan las circulares en sus teléfonos.

### **Requisitos funcionales:**

- RF1. El administrador debe autenticarse con usuario y contraseña.
- RF2. El administrador puede subir circulares en formato JPG, PNG o PDF.
- RF3. El sistema muestra automáticamente las circulares vigentes a todos los usuarios.
- RF4. La visualización se repite cada 4 horas durante 2 días desde la publicación.
- RF5. El sistema envía una notificación push al publicar una circular.
- RF6. El administrador puede previsualizar, desactivar/activar y eliminar circulares.

- RF7. El administrador consulta reportes: total enviadas, fecha, visualizaciones y estado.
- RF8. Los usuarios pueden hacer zoom sobre las circulares (imagen y PDF).

### Requisitos no funcionales:

- RNF1. Compatibilidad desde Android 8.0 (API 26) en adelante.
- RNF2. Interfaz limpia, moderna y usable (Material Design 3).
- RNF3. Disponibilidad y escalabilidad apoyadas en la infraestructura de Firebase.
- RNF4. Tiempo de difusión casi inmediato (sincronización en tiempo real).
- RNF5. Seguridad de acceso: solo el administrador puede publicar o gestionar.
- RNF6. Bajo consumo de recursos y de batería (tareas programadas eficientes).

## 3. Diseño del Proyecto

### 3.1. Diagramas de comportamiento

El siguiente diagrama de casos de uso resume las interacciones de los dos actores del sistema (administrador y usuario) con las funcionalidades principales.

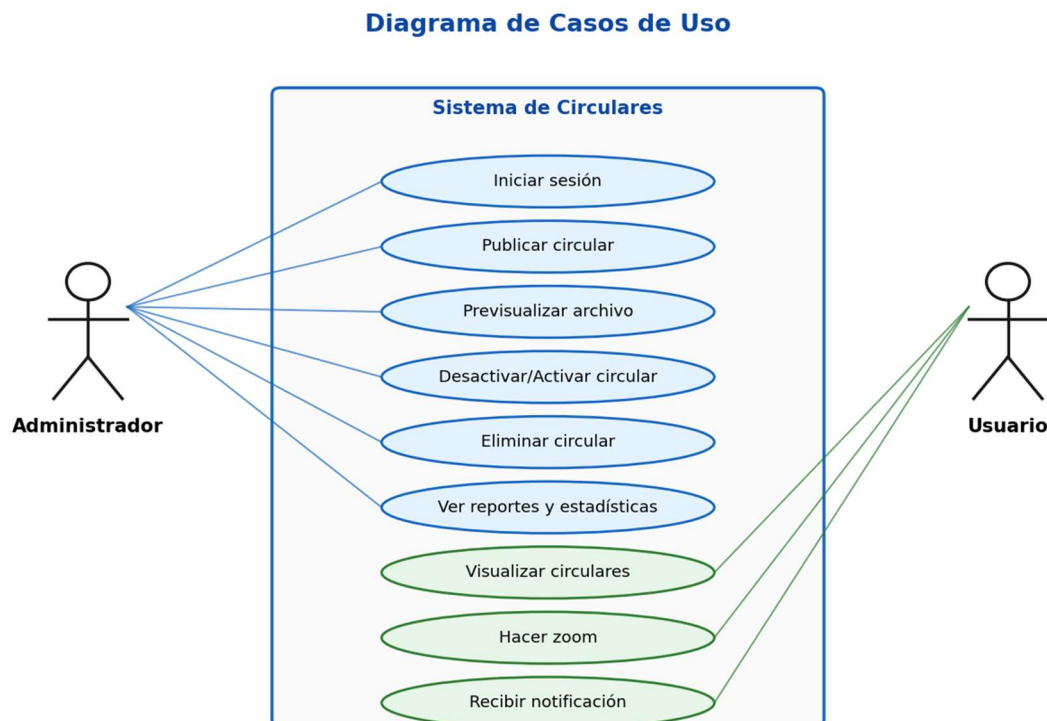


Figura 1. Diagrama de casos de uso del sistema.

El flujo de difusión describe el comportamiento del sistema desde que el administrador publica una circular hasta que esta se visualiza de forma repetida en los dispositivos de los usuarios.

### Flujo de Difusión de una Circular

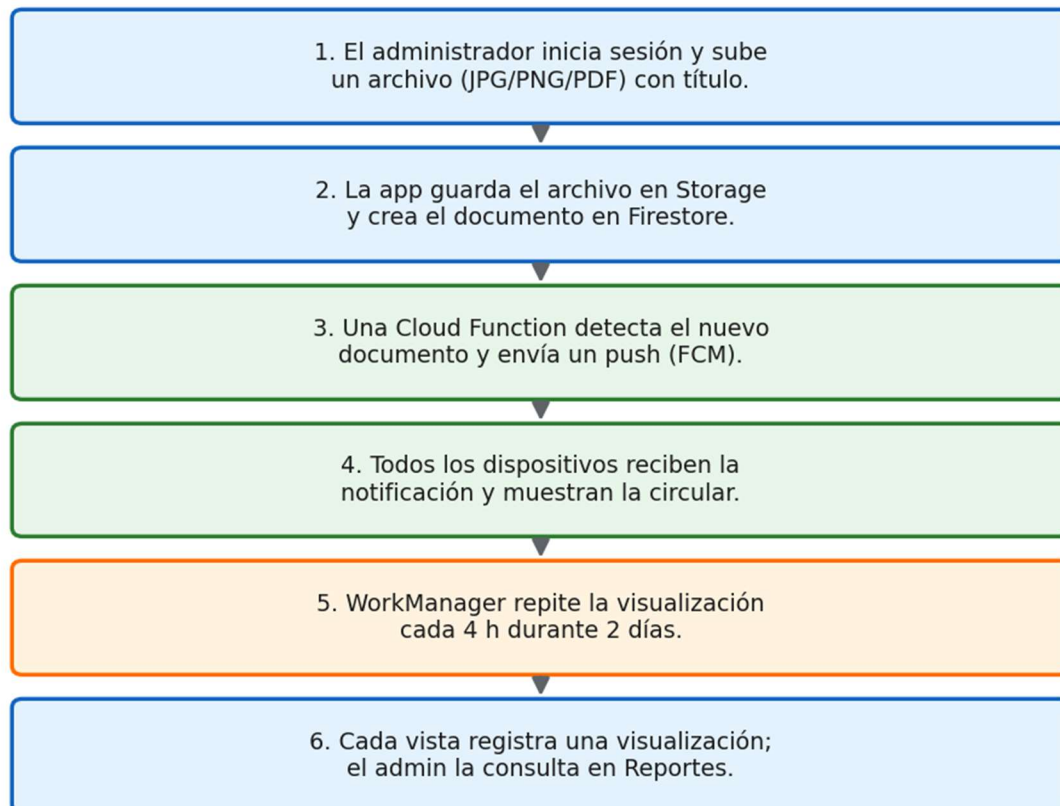
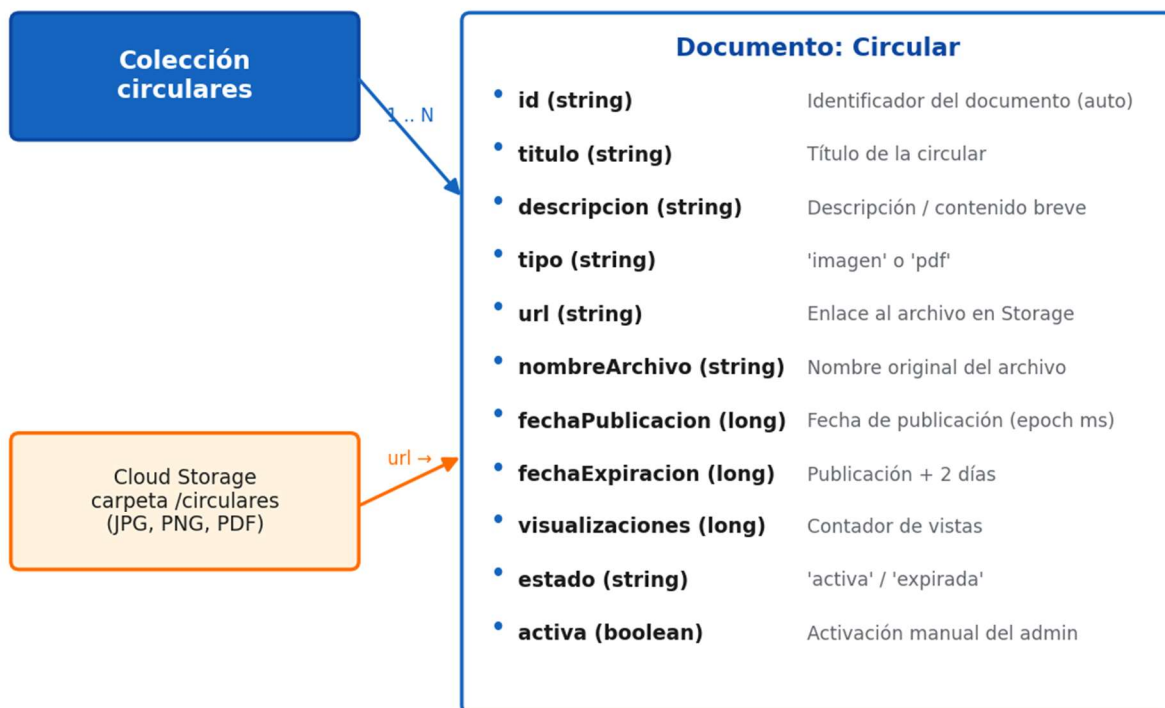


Figura 2. Flujo de difusión de una circular.

### 3.2. Modelo de Datos

El almacenamiento de datos se realiza en Cloud Firestore, una base de datos NoSQL orientada a documentos. El modelo se organiza en una colección «circulares», donde cada documento representa una circular con sus atributos. Los archivos binarios (imágenes y PDF) se guardan en Cloud Storage y se referencian desde el documento mediante una URL. Dado que Firestore no es relacional, el modelo entidad-relación se expresa como la relación entre la colección y sus documentos, y la vinculación de cada documento con su archivo en Storage.

## Modelo de Datos — Colección 'circulares' (Cloud Firestore)



*Nota: Firestore es NoSQL orientado a documentos; el 'modelo entidad-relación' se representa como colección → documentos con campos.*

Figura 3. Modelo de datos de la colección «circulares».

## 4. Arquitectura del Sistema

### 4.1. Descripción general de la arquitectura

El sistema adopta una arquitectura cliente-servidor con backend sin servidor (serverless / BaaS, Backend as a Service). El cliente es la aplicación Android, organizada internamente bajo el patrón MVVM (Model-View-ViewModel) para separar la interfaz, la lógica de presentación y el acceso a datos. El servidor está conformado por los servicios gestionados de Firebase (Google Cloud): Cloud Firestore como base de datos en tiempo real, Cloud Storage para los archivos, Cloud Functions para la lógica de notificación y Cloud Messaging (FCM) para el envío de mensajes push. Esta separación permite que la difusión sea automática: cuando el administrador escribe un documento, los demás clientes lo reciben por sincronización en tiempo real y, en paralelo, una función en la nube envía la notificación push.

### Diagrama de Arquitectura — Cliente / Servidor (Firebase)

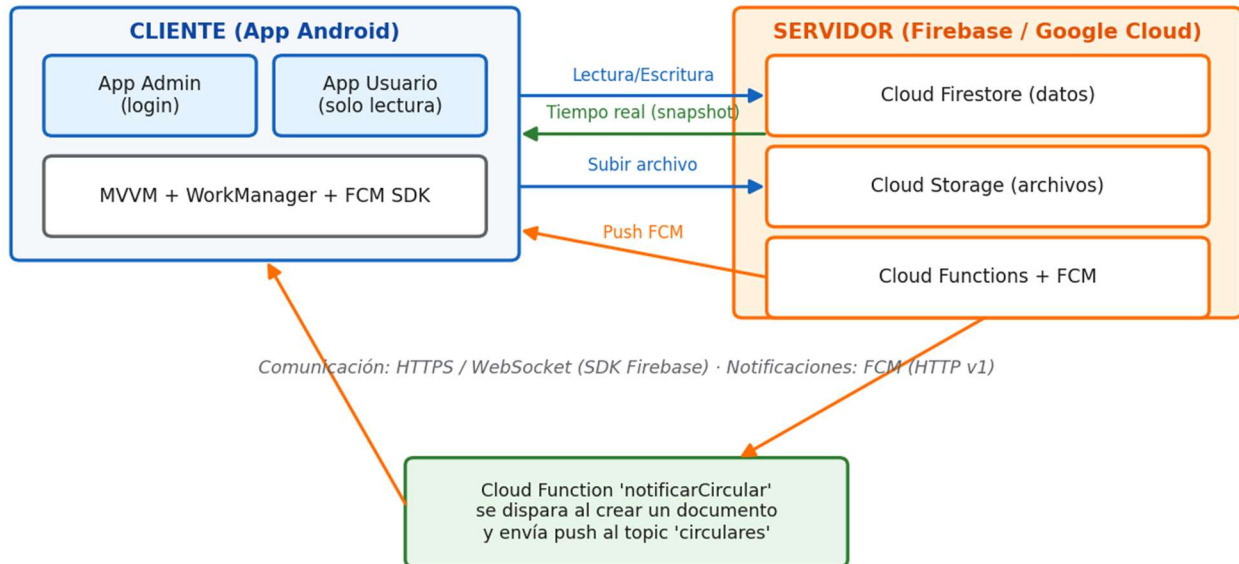


Figura 4. Diagrama de arquitectura del sistema.

## 4.2. Tecnologías utilizadas

| Componente                 | Tecnología                                                 |
|----------------------------|------------------------------------------------------------|
| Lenguaje                   | Kotlin                                                     |
| Arquitectura               | MVVM (ViewModel + LiveData) con Coroutines                 |
| Interfaz                   | Android SDK, Material Design 3, ViewBinding, Glide         |
| Base de datos              | Cloud Firestore (NoSQL, tiempo real)                       |
| Almacenamiento de archivos | Cloud Storage para Firebase                                |
| Notificaciones             | Firebase Cloud Messaging (FCM) + Cloud Functions (Node.js) |
| Tareas programadas         | WorkManager (repetición cada 4 h por 2 días)               |
| Compilación                | Gradle (Kotlin DSL), Android Studio                        |
| Compatibilidad             | Android 8.0 (API 26) hasta la más reciente (API 34)        |

### Protocolos de comunicación:

- HTTPS y WebSocket: utilizados por los SDK de Firebase para la lectura/escritura y la sincronización en tiempo real entre los clientes y Firestore/Storage.
- FCM (HTTP v1): protocolo de Firebase Cloud Messaging para entregar las notificaciones push a los dispositivos suscritos al «topic» de circulares.

## 5. Metodología Aplicada

---

### 5.1. Metodología de desarrollo

Se aplicó una metodología ágil de desarrollo incremental e iterativo, inspirada en los principios de Scrum. La justificación de esta elección radica en la naturaleza del proyecto: los requisitos podían refinarse a medida que se validaba el funcionamiento, y el equipo necesitaba entregar valor de forma temprana y continua. En la práctica, se construyó primero una «versión final» con todas las funcionalidades base operativas y, sobre ella, se incorporaron mejoras en una segunda iteración (eliminar y desactivar publicaciones, previsualización y zoom), lo cual ejemplifica el carácter incremental del proceso.

### 5.2. Ciclo de vida del proyecto

| Fase           | Actividades realizadas                                                             |
|----------------|------------------------------------------------------------------------------------|
| Planificación  | Definición del alcance, objetivos, requisitos y selección de tecnologías.          |
| Análisis       | Identificación del problema, actores, requisitos funcionales y no funcionales.     |
| Diseño         | Diseño de la arquitectura, del modelo de datos, de los diagramas y de la interfaz. |
| Implementación | Desarrollo en Kotlin con MVVM, integración con Firebase y WorkManager.             |
| Pruebas        | Pruebas funcionales de login, carga, difusión, notificaciones, reportes y zoom.    |
| Despliegue     | Generación del APK, despliegue de la Cloud Function y publicación del repositorio. |

### 5.3. Herramientas de gestión

- Git y GitHub: control de versiones y repositorio central del código fuente.
- Google Drive: almacenamiento y entrega de documentación del proyecto.

## 6. Seguimiento y Desarrollo

---

### 6.1. Planificación del trabajo (cronograma)

| Semana | Hito / Actividad                                             | Entregable              |
|--------|--------------------------------------------------------------|-------------------------|
| 1      | Planificación y análisis de requisitos                       | Documento de requisitos |
| 2      | Diseño de arquitectura y modelo de datos                     | Diagramas de diseño     |
| 3-4    | Implementación de módulos base (login, carga, visualización) | Versión final           |

| Semana | Hito / Actividad                                     | Entregable         |
|--------|------------------------------------------------------|--------------------|
| 5      | Integración de notificaciones (FCM + Cloud Function) | Push operativo     |
| 6      | Mejoras: eliminar, desactivar, previsualizar, zoom   | Versión 1.1        |
| 7      | Pruebas funcionales y correcciones                   | Reporte de pruebas |
| 8      | Despliegue, documentación y entrega final            | APK + informe      |

## 6.2. Roles y responsabilidades

| Integrante           | Rol y responsabilidad                              |
|----------------------|----------------------------------------------------|
| Wálter Guzmán Méndez | Desarrollo backend (Firebase, Cloud Functions)     |
|                      | Desarrollo Android (UI, MVVM, ViewModels)          |
|                      | Diseño de interfaz y pruebas (QA)                  |
|                      | Documentación y gestión (Git, cronograma, informe) |

## 6.3. Control de versiones

El control de versiones se realizó con Git, alojando el repositorio en GitHub. Se trabajó con la rama principal «main» y ramas de funcionalidad para cada mejora, integrando los cambios mediante confirmaciones (commits) descriptivas y, cuando correspondía, Pull Requests. El proyecto incluye un archivo .gitignore que excluye artefactos de compilación y las claves de firma.

**Importante (acceso del docente): se debe otorgar acceso total al repositorio de GitHub y a la carpeta de Google Drive del proyecto al correo institucional del docente: [americofiorilo.l@fcyt.umss.edu.bo](mailto:americofiorilo.l@fcyt.umss.edu.bo).**

## 6.4. Reporte de avances

El seguimiento del progreso se realizó mediante revisiones periódicas al cierre de cada iteración, contrastando lo desarrollado con los objetivos planteados. Cada avance (versión final 1 y versión 1.1) se registró en el repositorio y se verificó con pruebas funcionales antes de continuar con la siguiente etapa.

## 6.5. Comunicación

Dado que se realizó el proyecto por una sola persona no se necesitó ningún tipo de comunicación y/o coordinación ya que todas las tareas dentro del ciclo de desarrollo de cada aspecto de la aplicación más arriba mencionados y toda la documentación que conlleva el cumplimiento del proyecto fueron realizados por la misma persona.

## 7. Pruebas y Resultados

### 7.1. Plan de pruebas

Se diseñaron casos de prueba funcionales para verificar el cumplimiento de los requisitos. La siguiente tabla resume las pruebas realizadas y su resultado esperado.

| ID  | Caso de prueba                                       | Resultado esperado                                  |
|-----|------------------------------------------------------|-----------------------------------------------------|
| P1  | Inicio de sesión con credenciales válidas/ inválidas | Acceso solo con credenciales correctas              |
| P2  | Carga de circular en JPG, PNG y PDF                  | Archivo subido y documento creado                   |
| P3  | Visualización automática en otro dispositivo         | La circular aparece sin intervención                |
| P4  | Recepción de notificación push                       | Aviso recibido al publicar, app abierta o cerrada   |
| P5  | Repetición cada 4 h durante 2 días                   | Recordatorios dentro del periodo; cese al expirar   |
| P6  | Previsualizar antes de publicar                      | Vista previa correcta de imagen y PDF               |
| P7  | Desactivar / activar circular                        | Deja de mostrarse / vuelve a mostrarse              |
| P8  | Eliminar circular                                    | Documento y archivo eliminados                      |
| P9  | Zoom en imagen y PDF                                 | Acercamiento con pellizco y doble toque             |
| P10 | Reportes y estadísticas                              | Totales, fechas, visualizaciones y estado correctos |

### 7.2. Resultados obtenidos

En términos de funcionalidad, la aplicación cumplió con los requisitos definidos: el administrador publica circulares que se difunden automáticamente y se repiten durante el periodo establecido, y los usuarios las visualizan con posibilidad de zoom. En cuanto al rendimiento, la sincronización en tiempo real de Firestore permitió una difusión casi inmediata, y las notificaciones push se entregaron de forma confiable a los dispositivos suscritos. Respecto a la escalabilidad, el uso de infraestructura gestionada por Firebase permite atender un número creciente de usuarios sin administrar servidores propios, manteniendo el consumo dentro de la capa gratuita en escenarios de uso moderado.

### 7.3. Mejoras propuestas

- Vincular la apertura directa de la circular concreta al tocar la notificación (deep link con el identificador ya incluido en el mensaje).
- Integrar Firebase Authentication para reforzar la seguridad de escritura más allá del control local del administrador.
- Agregar categorías o áreas para segmentar la difusión por grupos de usuarios.

- Incorporar confirmación de lectura por usuario y reportes más detallados con gráficos.
- Soporte de modo sin conexión con caché local de las últimas circulares.

## 8. Conclusiones y Recomendaciones

---

El desarrollo del proyecto permitió construir una solución móvil funcional que resuelve la necesidad de difundir circulares e instructivos de manera automática, oportuna y medible. El uso de Kotlin con arquitectura MVVM facilitó un código organizado y mantenible, mientras que la plataforma Firebase aportó la sincronización en tiempo real, el almacenamiento y las notificaciones sin la complejidad de administrar servidores. La metodología incremental resultó adecuada, ya que permitió entregar primero una versión completa y luego enriquecerla con nuevas funciones.

El trabajo apoyado en el control de versiones, fue determinante para integrar los distintos componentes y cumplir los objetivos planteados.

### Recomendaciones para futuros desarrollos:

- Definir desde el inicio un esquema de seguridad basado en autenticación para los accesos de escritura.
- Automatizar la compilación y las pruebas mediante integración continua (por ejemplo, GitHub Actions).
- Realizar pruebas con usuarios reales para validar la usabilidad y la pertinencia de las notificaciones.
- Planificar el modelo de costos al escalar, considerando las cuotas de Firebase.

## 9. Bibliografía

---

Google. (2026). Documentación de Firebase. <https://firebase.google.com/docs>

Google. (2026). Cloud Firestore. <https://firebase.google.com/docs/firestore>

Google. (2026). Cloud Functions for Firebase (2.<sup>a</sup> gen.).  
<https://firebase.google.com/docs/functions>

Google. (2026). Firebase Cloud Messaging. <https://firebase.google.com/docs/cloud-messaging>

Android Developers. (2026). Guía de arquitectura de apps.  
<https://developer.android.com/topic/architecture>

Android Developers. (2026). WorkManager.  
<https://developer.android.com/topic/libraries/architecture/workmanager>

JetBrains. (2026). Kotlin Programming Language. <https://kotlinlang.org/docs/home.html>

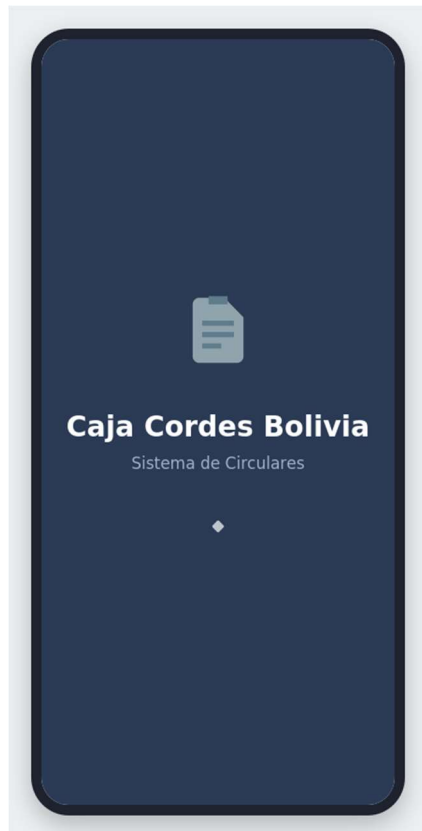
Google. (2026). Material Design 3. <https://m3.material.io>

## 10. Anexos

### Anexo A — Capturas de pantalla de la aplicación

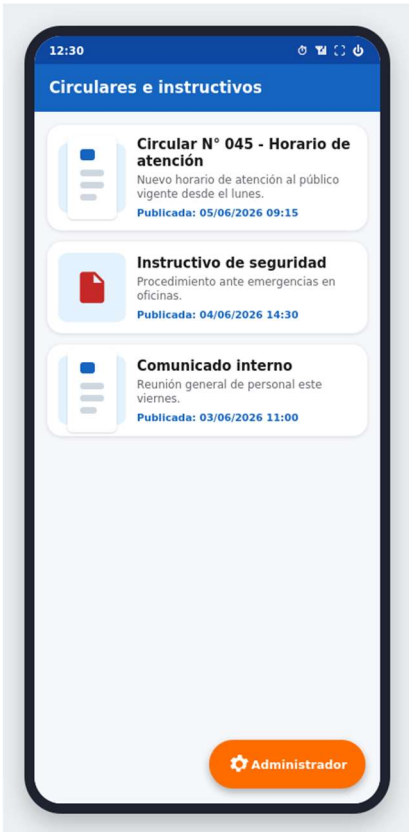
---

Proyecto «Caja Cordes Bolivia – Sistema de Circulares». A continuación, se presentan las pantallas principales de la aplicación, con una descripción de su función. Las imágenes representan la interfaz de la aplicación tal como fue diseñada (colores, componentes y distribución reales) y pueden sustituirse por capturas tomadas directamente del dispositivo una vez la aplicación esté en ejecución.



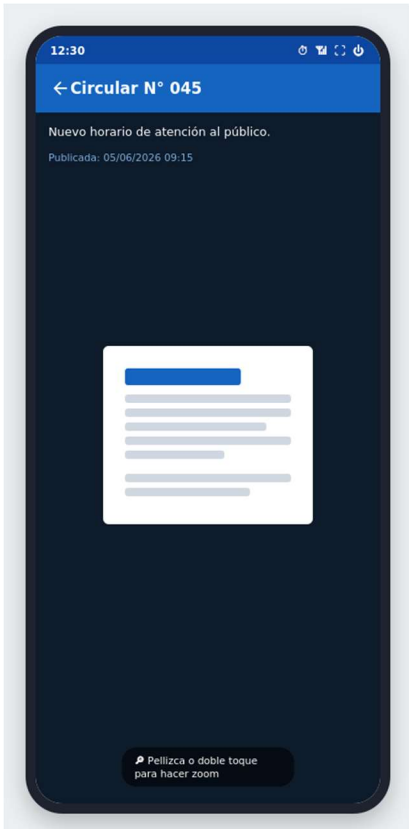
**Figura A.1. Pantalla de inicio (Splash)**

- Pantalla de bienvenida que se muestra al abrir la aplicación.
- Presenta la identidad del sistema: «Caja Cordes Bolivia – Sistema de Circulares».
- Carga inicial mientras se establece la conexión con Firebase.



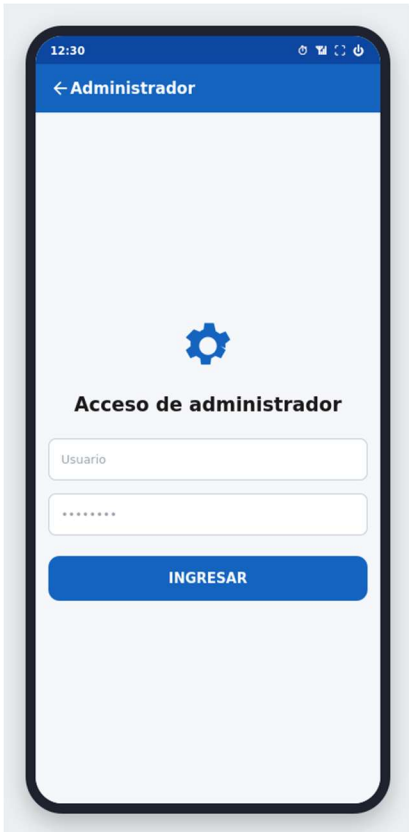
**Figura A.2. Pantalla principal (todos los usuarios)**

- Lista en tiempo real de las circulares vigentes; se actualiza automáticamente.
- Cada tarjeta muestra una miniatura (imagen o icono PDF), título, descripción y fecha.
- Botón flotante «Administrador» para el acceso del responsable.
- Los usuarios no necesitan iniciar sesión para ver el contenido.



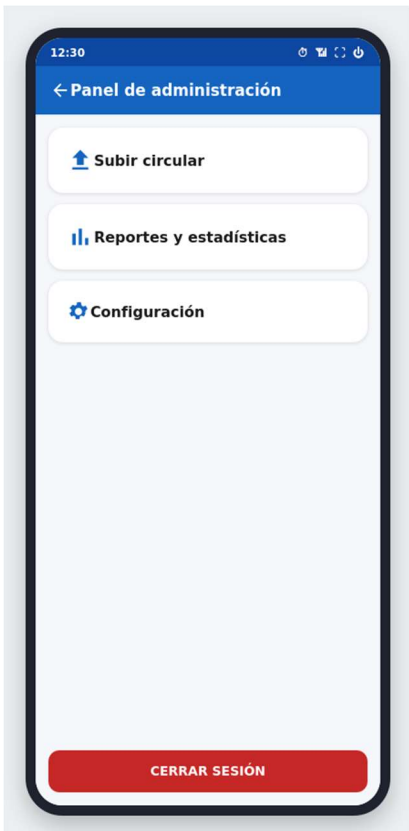
**Figura A.3. Visor de circular con zoom**

- Visualización a pantalla completa de la circular seleccionada.
- Permite acercar el contenido con pellizco (pinch) y doble toque.
- Funciona tanto para imágenes (JPG/PNG) como para documentos PDF.



**Figura A.4. Inicio de sesión del administrador**

- Acceso exclusivo del administrador mediante usuario y contraseña.
- Las credenciales se definen en el código o en la pantalla de configuración.
- Es la única parte de la aplicación que requiere autenticación.



**Figura A.5. Panel de administración**

- Menú principal del administrador una vez autenticado.
- Accesos a: subir circular, reportes y estadísticas, y configuración.
- Opción para cerrar la sesión de forma segura.



**Figura A.6. Carga de circular con vista previa**

- Formulario para publicar una nueva circular (título y descripción).
- Selección de archivo en formato JPG, PNG o PDF.
- Vista previa del archivo antes de publicarlo y botón «Abrir archivo».
- Al publicar, el archivo se sube a Storage y se difunde automáticamente.



**Figura A.7. Reportes y estadísticas**

- Tarjetas con totales: circulares enviadas, visualizaciones, activas y expiradas.
- Detalle por circular con su estado (ACTIVA / DESACTIVADA / EXPIRADA).
- Acciones de gestión: previsualizar, activar/desactivar y eliminar.
- Registro de la cantidad de visualizaciones y la fecha de envío.

